

UNIVERSITY OF BOTSWANA

FACULTY OF SCIENCE: COMPUTER SCIENCE DEPARTMENT

SUPERVISOR: Dr. Trust Mapoka

PROGRAMME: BSc Computer Science

Table of Contents

INTRODUCTION 3
- PROBLEM STATEMENT 3
- MOTIVATION 3
- PROPOSED SOLUTION 3
- PROJECT OBJECTIVES 3
- PROJECT SCOPE 4

METHODOLOGY 5
- DEVELOPMENT METHODOLOGY 5
- ARCHITECTURE AND DESIGN OF SYSTEMS 5
- COLLECTING REQUIREMENTS 5
- PHASES OF DEVELOPMENT 5
- METRICS FOR EVALUATION 6
- TECHNOLOGIES AND TOOLS 6

LITERATURE REVIEW 7
- FOUNDATION OF QUEUEING 7
- METHODS FOR QUEUE ANALYSIS 7
- OPTIMIZATION AND PERFORMANCE 8
- APPLICATION TO VIRTUAL QUEUE MANAGEMENT 8

RESOURCES 10
- HARDWARE 10
- SOFTWARE 10

SCHEDULE 11

IMPLEMENTATION 12
- OVERVIEW 12
- ARCHITECTURE 13
- BACKEND 14
- USER DASHBOARD 14
- STAFF DASHBOARD 15
- KNOWN IMPLEMENTATION LIMITATIONS 16

TESTING REPORT 17
- OVERVIEW 17
- TEST RESULTS 17

REFERENCES 21
SUMMARY 22

INTRODUCTION

PROBLEM STATEMENT

The finance office of the Universirty of Botswana tends to have a large volume of people queueing for help throughout the semester. This causes congestion at the office and it slows down the workflow as well as wasting the time of the people queueing for help.

MOTIVATION

The idea is to create a system that allows people to queue for help at the office without having to physically wait outside the office so that they can still get other things on their agenda done while also making it easier for staff members to deal with the workload they have.

PROPOSED SOLUTION

The solution being propsed is a system that can be used by anyone looking for help in any University of Botswana Administrative office and not just the finance offices to queue for help without being physically being present and only needing to show up when their time to be assited arrives. This system will be in the form of a web interface that can be accessed on the the University of Botswana's website by a person seeking assitance and another interface for the staff in the administrative offices to interact with the system while assisting the customer.

PROJECT OBJECTIVES

- To develop an easy to access online queuing system to reduce physical congestion at the administrative offices of the University of Botswana
- To improve the workflow of staff by reducing the time spent on managing a walk-in queue
- To allow individuals the ability to better spend their time by queueing remotely and only needing to show up when their time arrives

- To create a system that can be used through the whole institution and not just one particular office of section of the administration

PROJECT SCOPE

The goal of this project is to create a web-based queue management system with two interfaces for the administrative offices of the University of Botswana. The system will have an administrative interface for office staff to effectively manage and process queues, as well as a customer-facing interface that can be accessed via the university website, enabling staff and students to join lines remotely. The finance office will be the primary focus of the initial implementation, but the design will allow for future expansion to other university administrative departments. System design, development, testing, and deployment will all be covered by the project; however, hardware purchases and changes to the university's current infrastructure beyond software integration with the current website will not be included.

METHODOLOGY

DEVELOPMENT METHODOLOGY

Using iterative development cycles to guarantee ongoing feedback and improvement, the project will adhere to an Agile development methodology. Throughout the development process, this method enables adaptability to user requirements and technical difficulties.

ARCHITECTURE AND DESIGN OF SYSTEMS

A client-server architecture will be used in the system's design, and it will have two main web interfaces: one for customers and another for administrative staff. A database management system will be used by the backend to store user data, transaction logs, and queue information. To guarantee accessibility on a range of devices, including desktop computers, tablets, and smartphones, the frontend will be created utilising responsive web design principles.

COLLECTING REQUIREMENTS

Initial prerequisites will be gathered by:

- Interviews with finance office staff to understand current workflows and pain points
- Surveys distributed to students to gather user expectations and preferences
- Observation of current queuing patterns at the finance office during peak and off-peak hours
- Analysis of existing queue management systems used in other institutions

PHASES OF DEVELOPMENT

Phase 1: Prototyping and Design - Wireframes and UI mockups for staff and customer interfaces are created, and then prototypes are developed for user testing and feedback gathering.

Phase 2: Development of the Core System - User registration, queue joining, position tracking, notification systems, and staff queue management tools are among the crucial features that have been implemented.

Phase 3: Evaluation and Improvement - thorough testing that includes performance testing to make sure the system can manage anticipated user loads, unit testing, integration testing, and user acceptance testing with a sample group of employees and students.

Phase 4: Training and Deployment - Promotional materials to promote student adoption, training sessions for administrative staff, user documentation creation, and a pilot implementation at the finance office.Phases of Development

METRICS FOR EVALUATION

The effectiveness of the system will be evaluated by:

- decrease in office congestion and actual wait times
- Average wait time in comparison to the manual system that was in place before
- Staff and student user satisfaction surveys
- Statistics on system dependability and uptime
- Daily user count in comparison to baseline data

TECHNOLOGIES AND TOOLS

The development will make use of contemporary web technologies such as HTML5, CSS3, and JavaScript for the front end. The database system and backend framework will be chosen based on compatibility with the university's IT infrastructure. The system will be hosted on university servers or authorised cloud infrastructure, and Git will be used to manage version control.

LITERATURE REVIEW

FOUNDATION OF QUEUEING

Queuing theory provides the mathematical framework for analyzing waiting lines and service systems. Thomopoulos (2013) explains that queuing systems are characterized by several key components: the arrival process, the service mechanism, the queue discipline, and the system capacity. In the context of university administrative offices, these components translate to student arrivals, service time at help desks, the order in which students are served, and the physical or virtual capacity of the waiting area.

Arrival rate (λ) and service rate (μ) are the two basic parameters of any queuing system, and they work together to determine system performance metrics like average waiting time, queue length, and system utilisation (Thomopoulos, 2013). Designing a successful virtual queue management system requires an understanding of these parameters since they aid in forecasting how the system will behave under different load scenarios. According to Thomopoulos (2013), systems where λ approaches μ have noticeably longer wait times, highlighting the direct impact that the relationship between arrival and service rates has on customer satisfaction and operational efficiency.

METHODS FOR QUEUE ANALYSIS

In order to analyse and optimise queuing systems, statistical methods are essential. Thomopoulos (2013) talks about a number of probability distributions that are frequently used in queuing analysis, such as the exponential distribution for service times and the Poisson distribution for arrival processes. System designers can forecast system performance under various conditions and model real-world scenarios with the help of these statistical tools. Statistical analysis aids in capacity planning and resource allocation for university administrative offices, where arrival patterns may differ substantially between peak times (like registration weeks) and regular operations (Thomopoulos, 2013).

Statistical techniques are also used in performance evaluation, where metrics like average waiting time, average queue length, and system utilisation are computed and examined

(Thomopoulos, 2013). These metrics are crucial for both evaluating the success of a virtual queue management system after it has been deployed and for defending its deployment.

OPTIMIZATION AND PERFORMANCE

Anisimov (2009) and Thomopoulos (2013) both stress how crucial system optimisation is for queuing applications. Thomopoulos (2013) offers strategies for figuring out the ideal service capacity and striking a balance between the expenses of providing services and the costs of waiting times for customers. For university administrators who need to defend resource allocation choices, this economic viewpoint is helpful. According to the author, when the system runs close to capacity, even minor adjustments to service rate or queue management can result in significant drops in average waiting times.

Anisimov (2009) extends this discussion by examining dynamic queuing systems where parameters may change over time, which is highly relevant to university operations that experience seasonal variations in demand. The ability to model and predict these variations allows for proactive management strategies, such as adjusting staffing levels or implementing appointment systems during peak periods.

APPLICATION TO VIRTUAL QUEUE MANAGEMENT

A real-world implementation of queuing theory that overcomes many conventional constraints is the switch from physical to virtual queuing systems. According to Thomopoulos (2013), queuing systems can be made more efficient by decreasing the perceived waiting time even when the actual service time stays the same. By enabling users to do other things while they wait, virtual queue management successfully removes the psychological toll that physical waiting takes.

Additionally, Anisimov (2009) emphasises the significance of information transparency in queuing systems, pointing out that when customers have access to information about queue status and anticipated wait times, their perceptions of fairness and satisfaction levels improve. This psychological aspect of queuing that goes beyond mathematical models can be addressed

by a web-based queue management system, which can offer real-time updates on queue position and estimated service times.

RESOURCES

HARDWARE

- A computer for development
- Mobile device for testing

SOFTWARE

- IDE: Visual Studio Code
- Java: JDK 21+
- Maven:3.8+
- Node.js: 18+
- Npm: 9+
- DBMS: MySQL 8.0
- Web Development Libraries: React
- Version Control System: GitHub
- Project Tracker

SCHEDULE

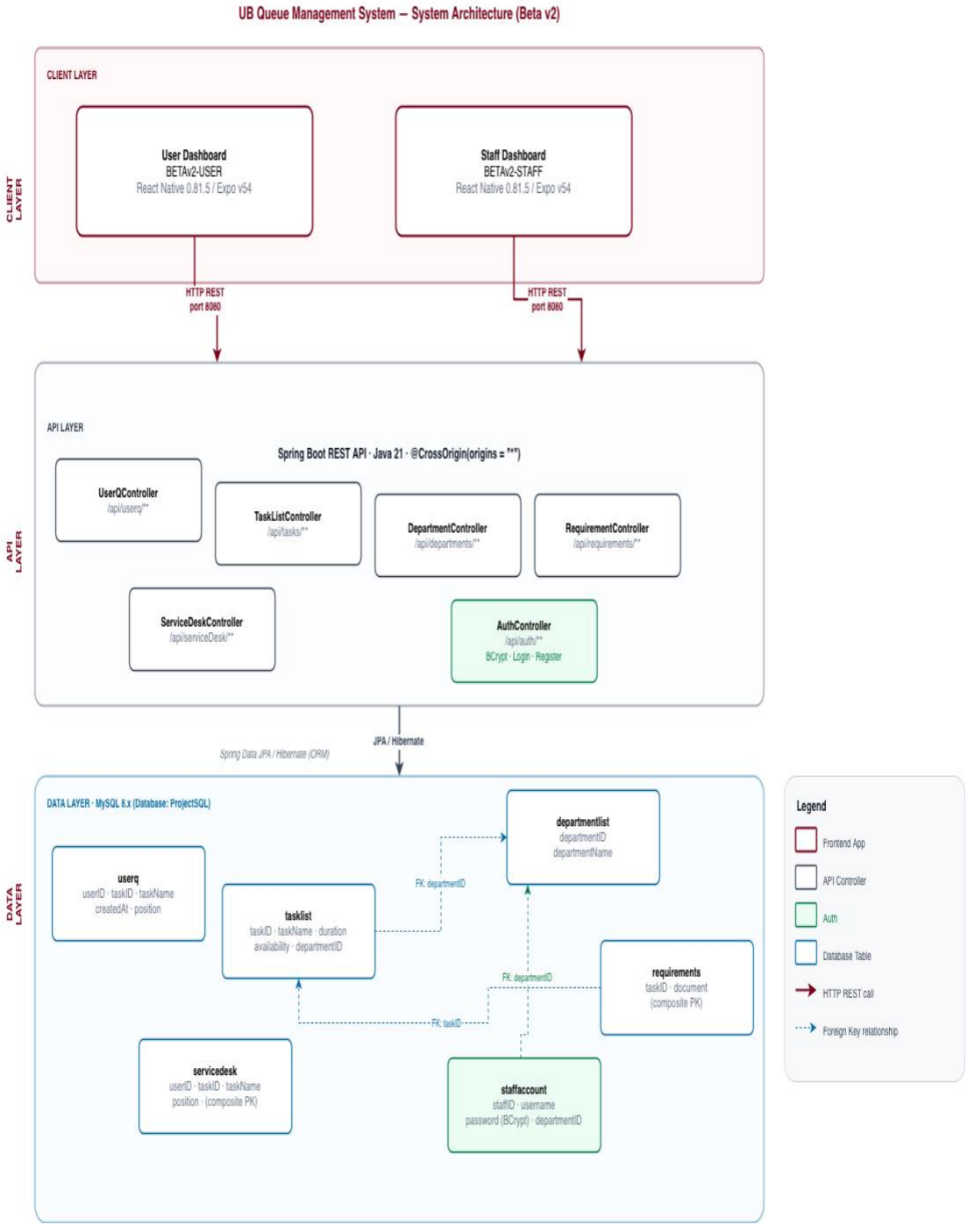
TASK	PROGRESS	START	END
Requirements Gathering and Analysis			
Staff Interviews	100%	2/12/26	2/15/26
Student Interviews	100%	2/15/26	2/17/26
Office Observation	100%	2/17/26	2/21/26
Design and Prototyping			
Wireframes	100%	2/18/26	2/23/26
Database Design	100%	2/20/26	2/25/26
Prototype Design	100%	2/26/26	3/3/26
Prototype Development	100%	3/4/26	3/12/26
System Development			
Frontend Dev	100%	3/14/26	3/28/26
Backend Dev	100%	3/29/26	4/12/26
Intergration	100%	4/13/26	4/27/26
Testing and Deployment			
Unit Testing	100%	4/29/26	5/6/26
Intergration Testing	100%	5/6/26	5/13/26
Deployment	90%	5/13/26	5/18/26

IMPLEMENTATION

OVERVIEW

The UB Queue Management System was implemented as a three-tier application comprising a Spring Boot REST API backend, a student-facing React Native mobile/web application (User Dashboard), and a staff-facing React Native mobile/web application (Staff Dashboard). All three tiers were developed and tested concurrently, with the backend exposing a consistent JSON API that both frontends consume over HTTP.

ARCHITECTURE



BACKEND

The backend is a Spring Boot 4.0.3 application written in Java 21, using Spring Data JPA with Hibernate as the ORM layer and MySQL 8 as the database. Maven is used for dependency management. The application exposes five REST controllers, each mapped to a dedicated resource path under /api;

- UserQ Controller
- TaskList Controller
- Department Controller
- Requirement Controller
- Auth Controller

Data models are defined as JPA entities with Lombok-generated getters and setters. Two entities use composite primary keys: Requirements (taskID + document) and ServiceDesk (userID + taskID), both implemented with a separate @Embeddable key class. The StaffAccount entity introduced in Beta v2 links each staff member to a DepartmentList record via a @ManyToOne relationship, allowing the login response to return department context directly. Passwords are stored as BCrypt hashes; plain-text credentials are never persisted.

The database schema is managed by JPA with ddl-auto=update, which creates or alters tables on application startup. A 'sample_data.sql' sql script is provided to populate all tables of the database with mock data.

USER DASHBOARD

The User Dashboard is a React Native 0.81.5 / Expo v54 application targeting both web and mobile. It is structured as a three-screen native stack navigator.

Component	**Implementation Detail**
UserWelcomeScreen	Landing page. Renders a split-panel layout on tablet/desktop and a centred card on mobile
JoinScreen	Fetches departments on mount, then tasks per department on selection. All tasks appear in the picker; unavailable ones show an

	inline warning banner. On submit, POST /api/userq/queue is called and the user is navigated directly to StatusScreen with the returned userID pre-loaded.
StatusScreen	Accepts an initialUserID route param for auto-load after joining. Fetches queue details and position in parallel. The cancel flow calls DELETE /api/userq/remove/{userID} and removes queue enttry.

STAFF DASHBOARD

The Staff Dashboard shares the same technology stack and navigation pattern. It consists of three screens navigated by a native stack.

Component	Implementation Detail
WelcomeScreen	Username/password login form. Calls POST /api/auth/login; on success, navigates to StaffDashboard passing departmentID, departmentName, staffID, and username as route params. Error messages from the API are displayed inline.
StaffDashboard	Receives department context from route params. Fetches /api/userq/department/{departmentID} on mount and via a 10-second setInterval for auto-refresh. Renders a FlatList of queue cards sorted ascending by createdAt. Each card shows a colour-coded position badge. A stats bar above the list shows waiting count, being-served count, and near-front count. On desktop the list renders in two columns.
ManageTasks	Loads all tasks and their requirements. Supports creating tasks (POST /api/tasks/add), updating (PUT /api/tasks/update/{id}), cascading delete (DELETE /api/tasks/delete/{id}), and adding/removing individual document requirements (POST /api/requirements/add, DELETE /api/requirements/delete/{taskID}/{doc}).

KNOWN IMPLEMENTATION LIMITATIONS

Several areas were intentionally deferred for post-beta resolution. The staff login validates credentials with BCrypt but issues no token, leaving API endpoints unprotected after login. The /api/auth/register endpoint has no admin guard. Queue user IDs are generated as random 5-digit integers, which could theoretically collide under high concurrency. All communication is plain HTTP with hardcoded localhost addresses in frontend constants. The ddl-auto=update JPA setting is suitable for development but should be replaced with a migration tool such as Flyway before any production deployment.

TESTING REPORT

OVERVIEW

This report covers testing done on Beta v2 of the queue management system. The main changes in this cycle were a full redesign of the User Dashboard UI, a new username/password login system for staff, and the addition of auto-polling and position alerts on the Status screen. All 42 test cases were run across the three system components -User Dashboard, Staff Dashboard, and the backend API.

Total Cases	Passed	Failed	N/A
40	**38**	**0**	1

39 tests passed. The 3 marked N/A were the live position alert cases (TC-USER-15, TC-USER-16) and the tablet layout test (TC-UI-05), which all require either a live multi-user queue or physical hardware that was not available during this round. No tests failed.

TEST RESULTS

User Dashboard

TC ID	Description	Result
TC-USER-01	Welcome screen loads -logo, title, and both buttons visible	**PASS**
TC-USER-02	Join the Queue button navigates to Join Screen	**PASS**
TC-USER-03	Check Queue Status button navigates to Status Screen	**PASS**
TC-USER-04	Joining a queue navigates directly to Status Screen with Queue ID pre-loaded	**PASS**
TC-USER-05	Join button stays disabled until a task is selected	**PASS**
TC-USER-06	Selecting an unavailable task shows a warning banner	**PASS**
TC-USER-07	Switching department clears and reloads the task list	**PASS**

TC-USER-08	Offline backend shows error message on Join Screen	**PASS**
TC-USER-09	Queue details display correctly when a valid ID is entered	**PASS**
TC-USER-10	Leave Queue removes the entry after confirmation	**PASS**
TC-USER-11	Non-existent Queue ID shows a not-found error alert	**PASS**
TC-USER-12	Tasks with no documents show "No specific documents required."	**PASS**
TC-USER-13	Queue ID field only accepts numeric input	**PASS**
TC-USER-14	Position refreshes automatically every 30 seconds	**PASS**
TC-USER-15	"Almost Your Turn" alert fires when position drops to ≤ 3	**N/A**
TC-USER-16	"You're Next!" alert fires when position reaches 1	**N/A**
TC-USER-17	After joining, Status Screen auto-loads with correct Queue ID	**PASS**

Staff Dashboard

TC ID	Description	Result
TC-STAFF-01	Valid credentials log in and open the correct department queue	**PASS**
TC-STAFF-02	Wrong credentials show an error -no navigation occurs	**PASS**
TC-STAFF-02B	Submitting empty login fields shows a validation message	**PASS**
TC-STAFF-03	Queue list auto-refreshes every 10 seconds	**PASS**
TC-STAFF-04	Manual Refresh button updates the list immediately	**PASS**

TC-STAFF-05	Serve / Remove removes the user from the queue	**PASS**
TC-STAFF-06	Empty queue shows "Queue is empty" state	**PASS**
TC-STAFF-07	Queue is sorted by join time -oldest entry first	**PASS**
TC-STAFF-08	New task is created and appears in the task list	**PASS**
TC-STAFF-09	Toggling availability updates the task globally	**PASS**
TC-STAFF-10	Adding a document requirement saves and displays correctly	**PASS**
TC-STAFF-11	Removing a document requirement removes it immediately	**PASS**
TC-STAFF-12	Deleting a task cascades to its requirements and queue entries	**PASS**
TC-STAFF-13	Only one task card expands at a time	**PASS**

Backend API

TC ID	Description	Result
TC-API-01	GET /departments/all -returns all 5 departments	**PASS**
TC-API-02	GET /tasks/department/{id} -returns only tasks for that department	**PASS**
TC-API-03	POST /userq/queue -creates entry with auto-generated 5-digit userID	**PASS**
TC-API-04	GET /userq/position/{id} -returns correct integer position	**PASS**
TC-API-05	DELETE /userq/remove/{id} -removes entry and returns 200 OK	**PASS**
TC-API-06	GET /tasks/all -returns all tasks with nested department objects	**PASS**

TC-API-07	GET /requirements/task/{id} -returns correct document list	**PASS**
TC-API-08	POST /auth/login -valid credentials return staff + department info	**PASS**
TC-API-09	POST /auth/login -wrong password returns 401	**PASS**
TC-API-10	POST /auth/login -missing fields return 400	**PASS**
TC-API-11	POST /auth/register -creates account with BCrypt-hashed password	**PASS**
TC-API-12	POST /auth/register -duplicate username returns 409	**PASS**

UI & Visual

TC ID	Description	Result
TC-UI-01	Serve / Remove and Delete buttons fill red on hover	**PASS**
TC-UI-02	Join Screen shows error/warning feedback for missing selections	**PASS**
TC-UI-02	Back to Home on Staff Dashboard returns to the login screen	**PASS**

REFERENCES

- Anisimov, V. (2009). Queueing Theory 1: Advanced Trends. ISTE Ltd and John Wiley & Sons Inc.
- Thomopoulos, N. T. (2013). Fundamentals of Queuing Systems: Statistical Methods for Analyzing Queuing Models. Springer Science+Business Media.

SUMMARY

This project focus on solving an issue of workflow efficiency in the University of Botswana for services that often have a high volume of people using them while also helping said people improve the way they manage their time when they plan to use the services of the university.